The Journal of Open Source Software

pymnet: A Python Library for Multilayer Networks

Tarmo Nurmi [1], **Arash Badie-Modiri** [2], **Corinna Coupette** [3,4], **and Mikko Kivelä** [1]

1 Aalto University, Finland **2** Central European University, Austria **3** KTH Royal Institute of Technology, Sweden **4** Max Planck Institute for Informatics, Germany

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Daniel S. Katz

Reviewers:

- @ClaudMor
- @pitmonticone
- @nwlandry

Submitted: 19 June 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Many complex systems can be readily modeled as networks and represented as graphs. Such systems include social interactions, transport infrastructures, biological pathways, brains, ecosystems, and many more. A major advantage of representing complex systems as graphs is that the same graph tools and methods can be applied in a wide variety of domains. However, the graph representation has its limitations. For many systems, there is a need to represent richer network data, such as multidimensional features, several types of interactions, numerous layers of hierarchy, and multiple modalities, to capture the structure and dynamics of the systems more accurately than simple graphs would allow. Multilayer networks (Kivelä et al., 2014) generalize graphs to capture the rich network data often associated with complex systems, allowing us to study a broad range of phenomena using a common representation, using the same multilayer tools and methods. With `pymnet`, we introduce a Python package that provides the essential data structures and computational tools for multilayer network analysis.

Statement of Need

`pymnet` is a Python package for creating, analyzing, and visualizing multilayer networks. It is designed for network scientists with an easy-to-use yet flexible interface, featuring, inter alia, representations of a very general class of multilayer networks, structural metrics of multilayer networks (e.g., clustering coefficients (Cozzo et al., 2015) and graphlet analysis (Sallmen et al., 2022)), multilayer-network transforms, multilayer-network isomorphisms and automorphisms (Kivelä & Porter, 2017) (with `bliss`; Junttila & Kaski (2011); Junttila & Kaski (2007)), and random multilayer-network models.

Different kinds of multilayer network data are becoming more and more available, but our computational tools for handling such data are lagging behind. Python is a popular programming language for network scientists and data scientists, and `pymnet` addresses the need for a feature-rich multilayer-networks package in the Python language that is actively maintained.

`pymnet` implements the general multilayer-network framework described by Kivelä et al. (2014). A *multilayer network* M is defined by $M = (V_M, E_M, V, \mathbf{L})$, where the sequence $\mathbf{L} = (L_a)_{a=1}^{d}$ defines sets L_a of *elementary layers*, the set V defines the *nodes* of the network, the *node-layers* are $V_M \subseteq V \times L_1 \times \ldots \times L_d$, and the *edges* $E_M \subseteq V_M \times V_M$ are defined between node-layers. Put simply, a node-layer is an association of a node $v \in V$ with a layer $\in L_1 \times \ldots \times L_d$ of dimensionality d, nodes can exist on an arbitrary number of layers, and edges can connect node-layers within layers and across arbitrary pairs of layers, which can differ in an arbitrary number of dimensions. The dimensions $1, 2, \ldots, d$ are called the *aspects* of the network.

Beyond the general multilayer-network framework described by Kivelä et al. (2014), `pymnet` also includes a specialized implementation of *multiplex* networks, a common subtype of multilayer

networks. In multiplex networks, edges across layers (interlayer edges) only occur between a node and its counterpart(s) on the other layers. The advantages of this specialization include, for example, automatic lazy evaluation of interlayer-coupling edges.

Main Features and Examples

pymnet's main data structure is `MultilayerNetwork`, which is implemented as a dictionary of dictionaries with a tensor-like interface, where each key represents a node, and each value is another dictionary containing information about the neighbors of each node, with the neighbors as keys and the weights of their incident edges as values. This structure ensures that adding nodes, removing nodes, querying for existence of edges, or querying for edge weights, all have constant average time complexity, and iterating over the neighbors of a node is linear in the number of nodes. Furthermore, the memory requirements are in $O(|V| + |L| + |E|)$ and typically dominated by the number of edges in the network.

To represent multiplex networks, pymnet offers `MultiplexNetwork`, which exploits the special structure of interlayer edges for efficiency, storing intralayer edges separately for each layer and only generating interlayer edges according to the applicable interlayer-coupling rules when they are explicitly needed. This ensures that we can always iterate over intralayer edges in linear time, and that interlayer edges only require constant memory (i.e., the memory to store the rule to generate them).

pymnet contains submodules for advanced analysis of multilayer networks. One example is graphlet-degree analysis, a powerful tool for investigating the structure of graphs that has been generalized to multilayer networks (Sallmen et al., 2022) and is implemented in pymnet for single-aspect multiplex networks. A graphlet is an isomorphism class of (connected) induced subgraphs that are typically small. pymnet can generate all graphlets of a specified size, i.e., all isomorphic multiplex networks with a user-specified number of nodes and layers (coming from a user-defined set of layers), user-defined interlayer couplings, and a user-defined type of multilayer isomorphism. From the graphlets, pymnet can compute the automorphism orbits of nodes or node-layers in the graphlets, with a user-specified type of isomorphism. For example, we can use pymnet to enumerate and visualize all automorphism orbits of nodes in single-aspect multiplex graphlets with two or three nodes and two layers under node-layer isomorphism. The results are depicted in Figure 1.

Other amenities shipped with pymnet include graph generators for generalizations of popular random-graph models to multilayer networks (e.g., Erdős-Rényi models and configuration models) as well as utilities for multilayer-network visualization.

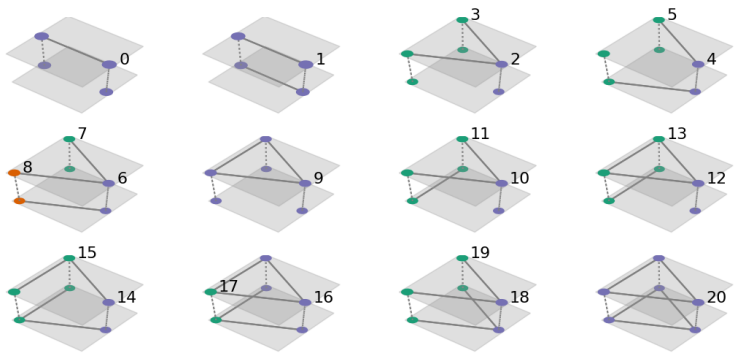

Figure 1: Using pymnet to enumerate and visualize automorphism orbits of nodes in single-aspect multiplex graphlets under node-layer isomorphism. Visualization script adapted from Sallmen et al. (2022).

DRAFT

Installation and Usage

Detailed installation and usage instructions, including tutorials demonstrating pymnet's main functionality, can be found in the pymnet documentation.

Related Packages

pymnet extends the popular networkx package developed for single-layer network analysis such that networkx functions can be applied to the individual layers of a multilayer network. These functions are automatically wrapped for use in pymnet, which has the benefit of automatically including new functionality added to networkx. To solve multilayer-network isomorphisms, pymnet uses a backend package, which can be either networkx (limited functionality) or bliss (Junttila & Kaski, 2011, 2007) (full functionality).

For multilayer-network visualization, pymnet uses matplotlib as its default backend, enabling users to exert low-level control over figure aesthetics to produce publication-quality plots. Support for interactive figures is provided via JavaScript and D3.js as a backend.

The only other library offering tools to work with multilayer networks in Python is multiNetX, which appeared after pymnet and seems no longer actively maintained. Support for working with multilayer networks in Julia is offered by MultilayerGraphs.jl from Moroni & Monticone (2023), who also compile a list of R packages offering similar functionality.

Projects Using pymnet

pymnet has been used in numerous scientific publications across different disciplines, such as Kivelä et al. (2014), Cozzo et al. (2015) Kivelä & Porter (2017), Danchev et al. (2019), Rio-Chanona et al. (2020), Zhou et al. (2020), Baek et al. (2021), Bergermann & Stoll (2021), Sallmen et al. (2022), and Nurmi & Kivelä (2023).

Acknowledgments

This work was supported by the European Commission FET-Proactive project PLEXMATH (Grant No. 317614), the Academy of Finland (Grant No. 349366), and Digital Futures at KTH.

References

Baek, E. C., Porter, M. A., & Parkinson, C. (2021). Social network analysis for social neuroscientists. *Social Cognitive and Affective Neuroscience*, *16*(8), 883–901. https://doi.org/10.1093/scan/nsaa069

Bergermann, K., & Stoll, M. (2021). Orientations and matrix function-based centralities in multiplex network analysis of urban public transport. *Applied Network Science*, *6*, 1–33. https://doi.org/10.1007/s41109-021-00429-9

Cozzo, E., Kivelä, M., De Domenico, M., Solé-Ribalta, A., Arenas, A., Gómez, S., Porter, M. A., & Moreno, Y. (2015). Structure of triadic relations in multiplex networks. *New Journal of Physics*, *17*(7), 073029. https://doi.org/10.1088/1367-2630/17/7/073029

Danchev, V., Rzhetsky, A., & Evans, J. A. (2019). Centralized scientific communities are less likely to generate replicable results. *Elife*, *8*, e43094. https://doi.org/10.7554/eLife.43094

Junttila, T., & Kaski, P. (2011). Conflict propagation and component recursion for canonical labeling. In A. Marchetti-Spaccamela & M. Segal (Eds.), *Theory and practice of algorithms in (computer) systems – first international ICST conference, TAPAS 2011, rome, italy,*

DRAFT

april 18–20, 2011. proceedings (Vol. 6595, pp. 151–162). Springer. https://doi.org/10.1007/978-3-642-19754-3_16

Junttila, T., & Kaski, P. (2007). Engineering an efficient canonical labeling tool for large and sparse graphs. In D. Applegate, G. S. Brodal, D. Panario, & R. Sedgewick (Eds.), *Proceedings of the ninth workshop on algorithm engineering and experiments and the fourth workshop on analytic algorithms and combinatorics* (pp. 135–149). SIAM. https://doi.org/10.1137/1.9781611972870.13

Kivelä, M., Arenas, A., Barthelemy, M., Gleeson, J. P., Moreno, Y., & Porter, M. A. (2014). Multilayer networks. *Journal of Complex Networks*, *2*(3), 203–271. https://doi.org/10.1093/comnet/cnu016

Kivelä, M., & Porter, M. A. (2017). Isomorphisms in multilayer networks. *IEEE Transactions on Network Science and Engineering*, *5*(3), 198–211. https://doi.org/10.1109/TNSE.2017.2753963

Moroni, C., & Monticone, P. (2023). MultilayerGraphs.jl: Multilayer network science in julia. *Journal of Open Source Software*, *8*(83), 5116. https://doi.org/10.21105/joss.05116

Nurmi, T., & Kivelä, M. (2023). *Subnetwork enumeration algorithms for multilayer networks*. https://doi.org/10.48550/arXiv.2308.00083

Rio-Chanona, R. M. del, Korniyenko, Y., Patnam, M., & Porter, M. A. (2020). The multiplex nature of global financial contagions. *Applied Network Science*, *5*, 1–23. https://doi.org/10.1007/s41109-020-00301-2

Sallmen, S., Nurmi, T., & Kivelä, M. (2022). Graphlets in multilayer networks. *Journal of Complex Networks*, *10*(2), cnac005. https://doi.org/10.1093/comnet/cnac005

Zhou, Y., Li, Z., Liu, Y., & Deng, F. (2020). Network proximity and communities in innovation clusters across knowledge, business, and geography: Evidence from China. *IEEE Transactions on Engineering Management*, *68*(5), 1388–1397. https://doi.org/10.1109/TEM.2020.3032160

DRAFT